2 DE OCTUBRE DE 2026

ETAPA 1: PLANEACIÓN DEL SOFTWARE - PROYECTO DE GESTIÓN (CHAT EMPRESARIAL)

HERRERA COQUE ANTHONY JHOEL, CHARRO JACOME MATEO NICOLÁS, MUELA GUACAPIÑA EDISON FRANCISCO
PROGRAMACION ORIENTADA A OBJETOS

1. Descripción del problema

El problema que presenta: La organización está fragmentada gravemente en su canal de comunicación interna. Los equipos de trabajo, en la actualidad, se coordinan en sus actividades cotidianas a través de aplicaciones de mensajería personal (WhatsApp), y correo electrónico disperso.

- Qué procesos, actividades o usuarios se ven afectados:

Procesos afectados: La coordinación operativa intersedes, el soporte técnico interno y el seguimiento de proyectos a corto plazo.

Usuarios con problemas: Todo el personal de administración, operación y los directores de área que pierden tiempo valioso buscando información dispersa entre múltiples plataformas.

- Qué consecuencias produce el problema:

Pérdida de conocimiento institucional: Los acuerdos y decisiones que se toman de forma verbal o en chats personales no quedan reflejados en los sistemas de la empresa.

Riesgos de seguridad: Compartir datos corporativos por vías no oficiales y susceptibles a vulneraciones.

Trazabilidad deficiente: Incapacidad para auditar el flujo de comunicación en caso de incidentes o requerimientos operativos.

- Por qué una aplicación desarrollada en Go podría contribuir a solucionarlo:

Es ideal ya que gracias a su gran eficiencia en el manejo de conexiones simultáneas en red y su muy bajo consumo de recursos computacionales. Esto permite construir un servidor de mensajería altamente receptivo, que sea capaz de mantener miles de conexiones concurrentes estables entre las sedes con una latencia mínima.

- Cómo se relaciona el problema con la operación entre las dos sedes:

La organización funciona de manera distribuida entre la Sede Principal (Quito) y la Sede Operativa (Sangolquí). El uso de canales informales o de conexiones no centralizadas produce retrasos, pérdida de sincronía y desconexión operativa entre ambas instalaciones. Una solución centralizada cliente-servidor estandariza la conectividad y los flujos de trabajo, independientemente de la ubicación geográfica.

- Delimitación del problema:

No se trata de que falte una aplicación sino de la vulnerabilidad, la ineficiencia y la falta de control institucional sobre el canal de comunicación e intercambio de datos operativos entre las sedes de la empresa.

2. Justificación del sistema que seleccionamos

Cuando una organización opera con equipos distribuidos en distintas sedes, la comunicación interna suele fragmentarse y perder trazabilidad. El propósito de este sistema es resolver ese problema de raíz: centralizar la mensajería instantánea en una plataforma segura y construir un historial formal de cada conversación.

- El sistema responderá a las necesidades de tres perfiles clave:

Personal operativo y administrativo: quienes requieren coordinarse en tiempo real durante la jornada.

Administradores de sistemas: encargados de la gestión de accesos, seguridad e infraestructura.

A nivel de uso diario, la plataforma facilitará el inicio de sesión seguro, la gestión de sesiones activas, el envío directo de mensajes y la consulta del historial de chats. Por debajo, administrará las credenciales de los usuarios (vía FastAPI), los registros de sesión, el contenido de los mensajes junto a sus marcas de tiempo (timestamps) y las métricas de uso de la plataforma.

- ¿Por qué elegimos una arquitectura cliente-servidor?

Principalmente para desacoplar responsabilidades y optimizar recursos. Un microservicio liviano construido en FastAPI se encarga de la seguridad y el control de acceso mediante tokens JWT, mientras que un motor de alto rendimiento en Go asume el trabajo pesado: manejar la concurrencia masiva y el flujo continuo de mensajes en tiempo real con la menor latencia posible.

3. Objetivos

- Objetivo General

Desarrollar una aplicación de chat empresarial con arquitectura cliente-servidor utilizando Go y un microservicio de autenticación en FastAPI, para agilizar la comunicación interna y respaldar la trazabilidad de la información entre las distintas sedes de la organización.

- Objetivos Específicos

Asegurar el control de acceso: Implementar un servicio de autenticación basado en tokens JWT con FastAPI para validar la identidad de cada usuario antes de conectarse.

Construir el núcleo concurrente en Go: Diseñar un servidor de red capaz de sostener múltiples conexiones simultáneas e intercambiar mensajes de texto de forma fluida.

Resguardar el historial de conversaciones: Estructurar el almacenamiento y la consulta de los chats para facilitar auditorías y mantener un registro confiable de la información institucional.

Monitorear la actividad del sistema: Generar métricas sobre el volumen de mensajes, las sesiones activas y los patrones de uso de los colaboradores.

Comprobar la calidad del software: Validar la estabilidad y el desempeño de la plataforma mediante pruebas unitarias e integrales en los componentes cliente-servidor.

4. Actores y usuarios

Actor	Descripción	Responsabilidades	Operaciones permitidas
Usuario Estándar	Colaborador de la empresa ubicado en la Sede Quito o Sangolquí.	Mantener comunicación fluida y profesional con sus compañeros.	Iniciar sesión, enviar y recibir mensajes, consultar historial de chat, ver lista de contactos activos.
Administrador / Supervisor	Personal de TI o gestión encargado de supervisar la plataforma.	Monitorear el buen funcionamiento del sistema y la seguridad de las sesiones.	Gestionar credenciales de usuarios, visualizar paneles de estadísticas y auditoría de mensajes.
Servicio Auth (FastAPI)	Microservicio externo encargado de la validación de identidad.	Procesar credenciales y emitir tokens de acceso seguro.	Validar usuarios, emitir y verificar tokens JWT.

5. Casos de uso

A. Cómo funciona el sistema en general

Todo empieza cuando el cliente le pide autenticación al microservicio en FastAPI, que le devuelve un token JWT. Con ese token en mano, el cliente se conecta al servidor principal en Go, donde puede intercambiar mensajes en tiempo real y revisar el historial guardado en la base de datos. Todo el proceso está pensado para que cada interacción sea segura.

B. Casos de uso principales

CU-01: Autenticación de usuario

Actor: Usuario estándar

Descripción: El usuario inicia sesión con sus credenciales corporativas para entrar al chat.

Flujo principal:

El cliente manda el usuario y la contraseña al microservicio FastAPI.

FastAPI comprueba que las credenciales sean correctas en la base de datos corporativa.

Si todo está bien, genera un token JWT y se lo devuelve al cliente.

El cliente guarda ese token para usarlo en las siguientes peticiones al servidor Go.

CU-02: Envío de mensajes en tiempo real

Actor: Usuario estándar

Descripción: El usuario puede escribirle un mensaje de texto a otro usuario que esté conectado al servidor Go en ese momento.

Flujo principal:

El usuario elige un contacto y escribe su mensaje.

El cliente envía el mensaje junto con el token JWT al servidor Go.

El servidor verifica el token, guarda el mensaje en la base de datos y se lo entrega al destinatario.

El sistema actualiza los contadores de actividad.

CU-03: Consulta del historial de conversación

Actor: Usuario estándar

Descripción: El usuario revisa, en orden cronológico, los mensajes que ha intercambiado con un contacto.

Flujo principal:

El usuario elige el contacto cuyo historial quiere ver.

El cliente le pide al servidor Go los mensajes anteriores, enviando el token JWT.

El servidor confirma que el usuario tiene permiso, consulta la base de datos y devuelve los mensajes ordenados del más antiguo al más reciente.

6. Matriz de trazabilidad inicial

Necesidad identificada	Objetivo relacionado	Código del requisito	Caso de uso	Criterio de aceptación
Falta de seguridad en el acceso a canales internos	Objetivo 1	RF-01	CU-01	El sistema rechaza credenciales inválidas y emite un JWT válido ante credenciales correctas.
Retrasos y desconexión intersedes por falta de tiempo real	Objetivo 2	RF-02 / RF-03	CU-02	Los mensajes se entregan al destinatario en menos de 1 segundo de forma concurrente.
Pérdida de acuerdos institucionales en chats informales	Objetivo 3	RF-04	CU-03	El historial se muestra completo, ordenado cronológicamente y accesible solo por los involucrados.
Ausencia de métricas sobre el uso del canal interno	Objetivo 4	RF-05	Panel Admin	El sistema genera un reporte estadístico con el volumen de mensajes por franja horaria.

7. Conclusiones

Viabilidad de la aplicación: El proyecto es muy viable. Go ofrece un rendimiento sobresaliente para manejar muchas conexiones concurrentes, y FastAPI aporta una base sólida para la autenticación con estándares modernos como JWT.

Coherencia entre el problema y los requisitos: Hay una relación clara y directa entre los problemas detectados en la comunicación entre sedes, los objetivos planteados y los requisitos funcionales definidos en el alcance.

Principales retos en el diseño y desarrollo con Go: Los más importantes serán gestionar bien la concurrencia en conexiones persistentes (como los WebSockets), mantener sincronizados de forma segura los estados de sesión y manejar de manera eficiente los errores de red entre las distintas sedes.

Decisiones pendientes para la siguiente fase: Habrá que definir el esquema final de la base de datos relacional, la estructura de paquetes siguiendo una arquitectura limpia (clean architecture) en Go, y el protocolo exacto de comunicación en tiempo real entre el cliente y el servidor.